Temporal

What does an agent need to remember?

It can remember the conversation and forget the work.

Melanie Warrick · AI & DevRel @ Temporal

WHAT JUST HAPPENED

A memory store could bring back the answers. Could it bring back the work?

- It doesn't know if the refund went out.
- It couldn't tell which step runs next.
- Nothing would restart the run.

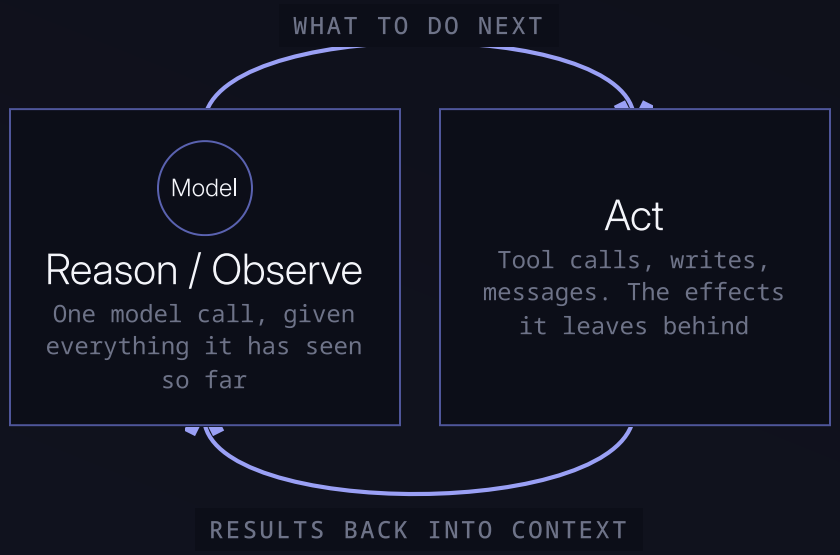
Stripe knew the payment went through. Nothing kept track of where the work stood.



THE LOOP

An autonomous agent is a loop

It is literally a loop. Plain Python, no SDK, no magic.



The result is just context for the next model call. It repeats until it reaches a goal, a stop condition, or a failure.

WHERE AGENTS SIT

AI agency levels

Level 1

Programmatic

Control flow entirely in code.

Easy to reason about program behavior.



Level 2

Guided

Control flow mostly in code. LLM makes limited decisions.

May require human review.



Level 3

Agentic

The LLM makes high-level control flow decisions.

Observability and guardrails are critical.

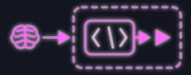


Level 4

Code-Gen

The LLM actively writes and executes code.

Requires sandboxing and other controls.



Level 5

Self-Evolving

The agent harness can modify its own code.

Needs bulletproof isolation and audit.



Level 3+: path chosen at runtime → can't pre-declare states

AROUND THE LOOP

Only context lives inside the loop

IN THE LOOP

CONTEXT

The messages list, rebuilt every turn



OUTSIDE THE LOOP

LONG-TERM MEMORY

What carries across runs, in a store

OUTSIDE THE LOOP

STATE

Facts the model does not own

The loop holds context. Who holds memory and state?

CONCRETELY

What goes where

One autonomous support agent, with authority to issue a refund.

CONTEXT

What the model sees this turn

- The customer's message
- The order it just looked up
- The last few tool results

LONG-TERM MEMORY

What it carries across runs

- The refund policy
- This customer's past cases
- What went wrong last time

STATE

Facts the model does not own

- Which tools already ran
- The refund it issued
- Blocked waiting on approval

Lose the first two and the agent repeats work. Lose the third and it refunds the same order twice.



THE FAMILIAR TAXONOMY

Memory comes in kinds

WORKING

Like a whiteboard.

What is active for the current decision.

`The messages sent to the model`

EPISODIC

Like a diary.

Past experiences and events.

`Run logs, transcripts`

SEMANTIC

Like an encyclopedia.

Facts as the agent recalls them.

`Vector store, knowledge base`

PROCEDURAL

Like a recipe.

Skills, and how to do things.

`Skill files, tool definitions`

PARAMETRIC Like what you already knew. Baked into the weights · can't edit while running

---

All of it answers one question: what should the agent know?



CONTEXT MANAGEMENT

Managing what the model sees

The window is a buffer. Context prep is what makes it behave like working memory.

01	Stuff the window	Everything in the prompt
02	Truncate	Drop the oldest turns
TODAY'S DEFAULT 03	Compact	Summarize old turns
04	Retrieve	RAG, memory files, memory tools
SHIPPED 05	Bigger windows	A million tokens, already here
EMERGING 06	Self-curating context	The agent edits its own playbook

Long-running loop → context grows every turn → so does the history

Manages what it knows → not where the work stands



PARTS OF THE SYSTEM

Facts have owners. Each owner is a system of record

Temporal owns one lane → read the rest live

TEMPORAL'S LANE

Execution state

`Temporal, in this demo`

- Which steps already ran
- The attempt, and its identity
- Where the loop is blocked

Authorization state

`Your auth system`

- May this agent act at all
- Scope of what it may touch
- Revoked since it started

Effect state

`Stripe, your APIs`

- Did the charge clear
- Did the refund actually post
- Attempted, or committed

Domain state

`Your database`

- The refund row
- The customer's plan
- The ticket's status

The test: does another system already own this fact?



WORKED EXAMPLE

No single system knows the whole refund

Order 1234, at a single moment in the run.

WHAT THE AGENT SEES Context	"Refund order 1234 for $80."
WHAT THE AGENT REMEMBERS Long-term memory	"Loyal customer. We refunded them once before." Useful, but it may be out of date.
WHAT HAPPENED Effect state	Stripe: Refund succeeded.
WHERE THE WORK STANDS Execution state	Temporal: Refund step still in progress.

First two → decide · last two → recover safely

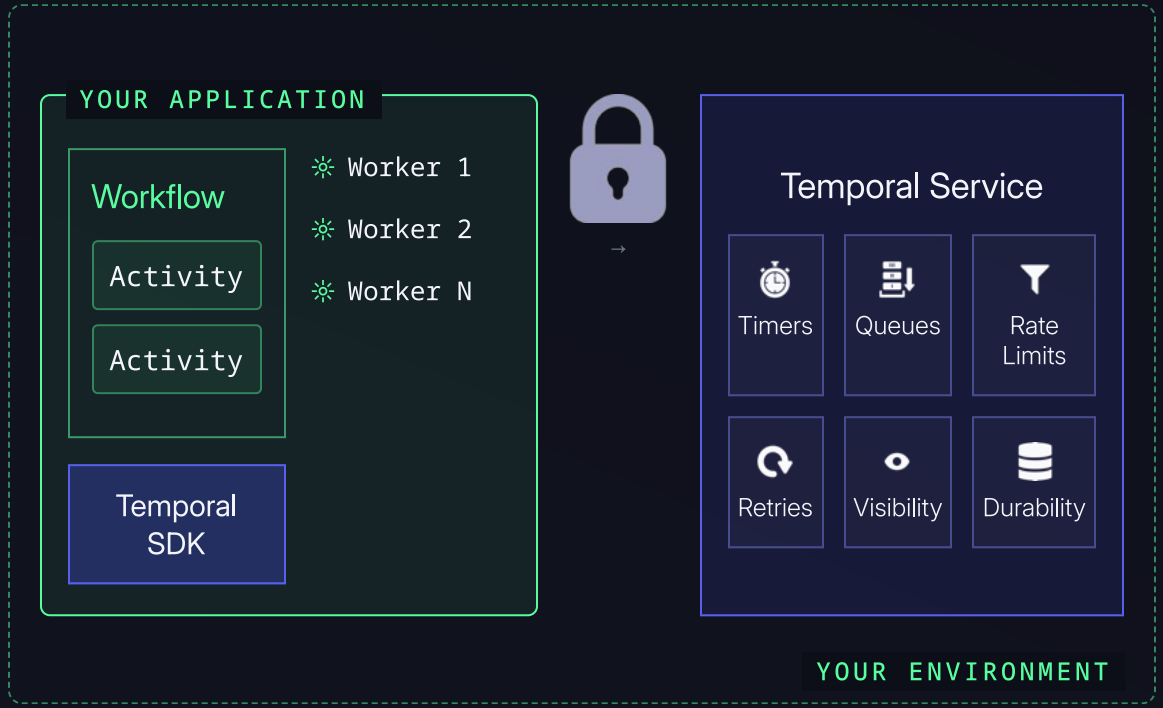
Disagree → the record wins, or you refund twice



THE ENGINE UNDERNEATH

What is Temporal?

- Build crash-proof applications
- Open source, MIT license
- Write in regular code: Python, TypeScript, Java, Go, Rust, Ruby, .NET
- Uses *memoization* and *replay* to recover from failures at any point



TEMPORAL · THE VOCABULARY

Core Temporal concepts

WORKFLOW

your function · the Temporal Service records its steps

`RefundApprovalAgent`

ACTIVITY

One call that leaves the process. Retried on failure.

`agent_decide_next_step`, `issue_refund`

WORKER

The process that runs your code. Disposable.

`your Python process`

TASK QUEUE

How work reaches a Worker. Workers poll it.

`refund-stage-<id>`

EVENT HISTORY

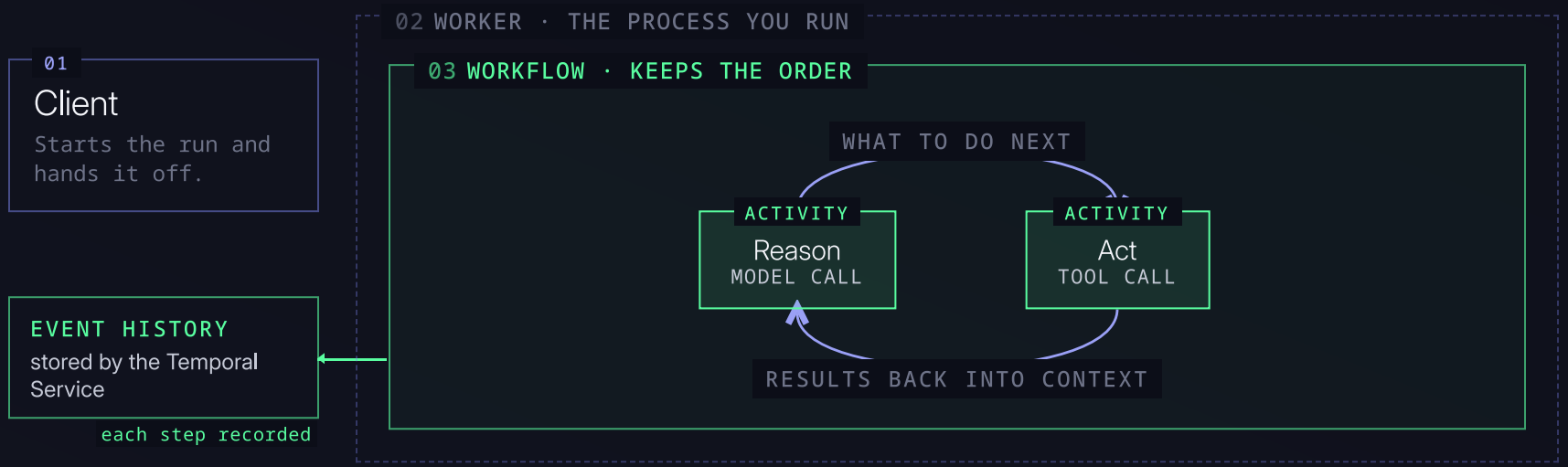
The record of what already finished.

`order looked up`, `policy checked`

You write Workflows and Activities. Temporal keeps the history, and Workers do the work.

THE SHAPE IN A REAL SYSTEM

Where the durable loop runs



Worker stops → Workflow stays Running, keeps loop + state

HOW IT WORKS

Durable execution reconstructs state from recorded history



Replay, not redo.

- Demo: lookups + decision read back → only the refund ran
- Working memory in Workflow state → back too
- Long-term memory → still your store
- Same idempotency key every retry → Stripe makes one refund

HOW IT WORKS

Replay, not redo

NEW WORKER

WORKFLOW CODE (SIMPLIFIED)

turn - ▶ Play

```
for turn in range(MAX_TURNS):
    step = await execute_activity(agent_decide_next_step, working_memory)
    if step.action == "decide":
        break
    if step.action == "ask_customer":
        await wait_condition(answered)        # customer_answer Signal
    else:
        working_memory.append(await run_tool(step.tool))
refund = await execute_activity(issue_refund, step)
```

EVENT HISTORY · TEMPORAL SERVICE

1 Agent turn 1–5

2 customer_answer ×2

3 lookup_order

4 lookup_customer_history

New Worker runs the code from the top. Finished steps return recorded results. Only the next step runs.

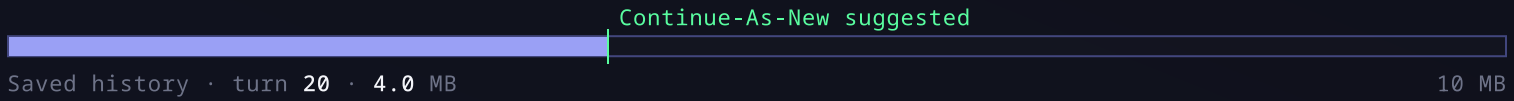


GOTCHA

Every step is saved, so long agent runs build a big history

Won't the saved history get too big?

- Temporal saves every step → after a crash, a new Worker replays it to rebuild the agent.
- Every turn re-sends the whole chat to the model → the history grows faster than the chat.
- One result too big (Temporal's 2 MB limit) → Fix 1: claim check.
- A run that goes on and on (Temporal's 50 MB limit) → Fix 2: Continue-As-New.

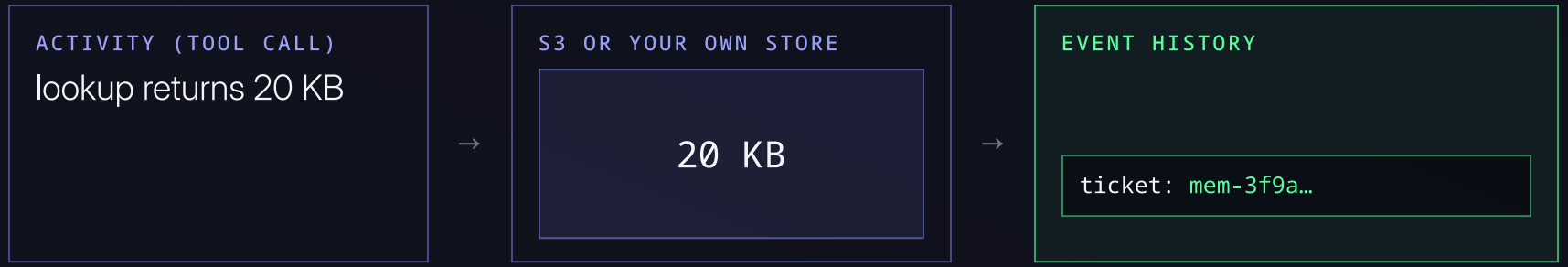


ESTIMATE Example: 20 KB results → turn 20 alone saves 400 KB. About 4 MB total.

FIX 1

Fix 1 · Claim check: save a ticket, not the data

Problem: one tool result is too big to save.



Like a coat check: Temporal holds the ticket.

- Needed if the agent pulls a customer's full order history. This demo's results are small.
- Temporal's Python SDK can do this for you (preview).

It shrinks the history, not the prompt.

FIX 2

Fix 2 · Continue-As-New: start a fresh history, same Workflow

Problem: the run goes on so long the history piles up.

Run 1

Workflow ID refund-1234
Run ID a1b2… (one per run)

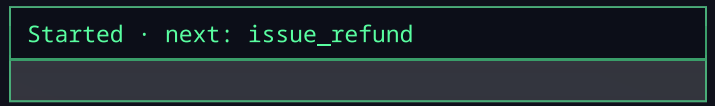


→

YOU CARRY OVER
order 1234 · $80
lookup ✓ · policy ✓
2 answers
next: issue_refund

Run 2

Workflow ID same
Run ID new · c3d4…



Like a new bank statement: it opens with your balance, not every past transaction.

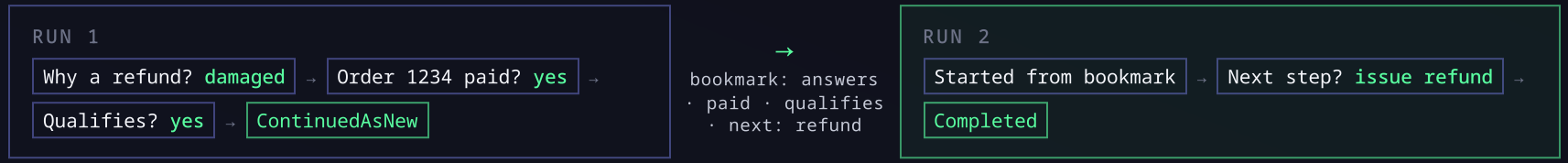
> Carry the state exactly. Summarize the memory.

FIX 2 · IN CODE

Continue-As-New in code (sketch)

SKETCH · NOT RUN IN THE DEMO

```
if workflow.info().is_continue_as_new_suggested():          ← history is getting big
    await workflow.wait_condition(workflow.all_handlers_finished)   ← save any answer still arriving
    bookmark = self.where_the_work_stands()                 ← steps done · answers · next step · refund key
    workflow.continue_as_new(replace(request, resume=bookmark))     ← same Workflow ID, fresh history
```



Saved history: 3,461 → 554 bytes

The refund's duplicate-check key uses the Run ID → carry it, or roll over before the refund

THE TRADE-OFF

Not every agent needs durable execution

YOU MIGHT NOT

- short loop
- the user is right there
- a lost run means asking again
- your runtime already resumes runs

IT EARNS ITS PLACE

- work outlives a process
- long waits on people or systems
- retries against real side effects
- deploys mid-task

Its costs: history holds customer data, so plan retention and encryption. Someone runs the service, or pays for Cloud.

> Temporal won't make the model's choice right → that's evals + guardrails.



COST TO RUN

A restart pays for every model call again

MEASURED OCT 2 · ONE PASS	4 model calls · 2,962 tokens · $0.0009 (under a tenth of a cent)
Without Temporal INTERRUPTED, START OVER	Same 4 calls again: 2,972 tokens, $0.0009 more
With Temporal INTERRUPTED, RESUME	0 calls re-sent. Finished calls replay from Event History.
AT SCALE ESTIMATE	An agent serving 1M requests a month: ≈3B tokens, ≈$900 in model calls. Every request that starts over pays its share again.

A call in flight when the process dies can be billed again.
List prices: Sep 29



THE DISTINCTION

REASONING	Context	What the agent can see right now	
REASONING	Long-term memory	What the agent remembers or looks up	
THE OPERATION	Execution state	Where the work stands	TEMPORAL
THE OPERATION	Effect state	Whether money moved	STRIPE

Memory helps reasoning continue. Temporal helps the operation continue.



TRY IT

Run it yourself

```
$ make setup && make run
```

no keys · needs uv + the Temporal CLI

REPO

github.com/temporal-community/temporal-ai-agent-memory-state-stripe

RUN TEMPORAL LOCALLY

```
$ brew install temporal
$ temporal server start-dev
```

TEACH YOUR CODING AGENT TEMPORAL

```
$ npx skills add temporalio/skill-temporal-developer
```

Temporal docs
docs.temporal.io

Payloads too big: External Storage
docs.temporal.io/develop/python/data-handling/external-storage

More in the Temporal & AI Series
Durable human-in-the-loop agents, multi-agent handoffs

History too long: Continue-As-New
docs.temporal.io/develop/python/continue-as-new

Stripe idempotent requests
docs.stripe.com/api/idempotent_requests

TAKEAWAYS

Temporal delivers Durable Execution

TEMPORAL CUSTOMER RESULTS

Faster velocity

Deliver new features ~10x faster.

NEW FEATURES

60–80 days → 5–10 days

MAERSK

Improved reliability

Fewer errors, less lost revenue.

PAYMENTS

99.4% → 99.99%

LARGE FINANCIAL INSTITUTION

Full observability

See every step of systems and agents.

DEPLOYS FAILED BY CLOUD ERRORS

4% → 0.0001%

NETFLIX

Sources: Maersk and Netflix public case studies · payments figure from a large financial institution

TAKEAWAYS

For agents, it pays off twice

COST

Reduces model calls and costs

AUTONOMY

Enables longer-running workflows and loops

